TERMAGUARD.AI | FIRMWARE & HARDWARE INTEGRATION

ESP32 4G Telemetry Packet Specification

Edge Node HTTP Communication Protocol, Sensor Data Dictionary & Firmware Implementation Guidelines (Release V1.0)

Doc Reference:	TG-SPEC-FW-4G-V1.0	**Date:**	October 2026
Target Hardware:	ESP32 + 4G LTE Modem	**Target Backend:**	Spring Boot REST API

1. System Overview & Network Transport Protocol

The TermaGuard edge node is an ESP32-based autonomous sensor node deployed across commercial tree plantations. It communicates directly with the central cloud/edge gateway via a 4G LTE-Cat1 or Cat-M1 cellular modem (e.g. SIMCom SIM7600/A7670 or Quectel EC200U) over a standard HTTP POST REST interface.

Transport Protocol	HTTP/1.1 POST over TCP/IP 4G Cellular Link
Target Ingestion URL	`http://<SERVER_IP>:8080/api/v1/telemetry/uplink`
Payload Content-Type	`application/json`
Authentication Header	`X-Node-API-Key: termiguard-node-secret-2026`
ESP32 User-Agent	TermaGuard-Node/1.0 (ESP32-4G)
Server Response Code	200 OK with parameter synchronization JSON body

2. Operating Modes & Edge State Machine

To maximize battery endurance over multi-year field deployments, the firmware employs an environmental gating strategy: subterranean termites require high humidity and moist soil (>65%) to construct shelter mud tubes. When soil is dry, acoustic circuits are gated off.

[MODE 1: ACTIVE 15-MINUTE TRIGGERED SAMPLING] Awakens every 15 minutes. Triggered when Soil Moisture >= 65.0%. The ESP32 powers up the analog piezo acoustic circuit via a low-dropout switch, records a 5-10s audio window, calculates the acoustic energy score (0-100) and 13-band Mel-Frequency Cepstral Coefficients (MFCC), transmits the full payload over 4G, and returns to deep sleep for 15 minutes.

[MODE 2: 24-HOUR DORMANT HEARTBEAT (KEEPALIVE)] Awakens once every 24 hours (or configurable heartbeat interval). Triggered when Soil Moisture < 65.0%. Termites do not forage in dry soil; hence the acoustic sensing circuit is held in hard power-down to preserve battery. The node sends battery voltage, battery percentage, microclimate, and cellular link metrics (RSSI/SNR). Fields 'acousticScore' and 'mfccFeatures' MUST be set to null. Deep sleep resumes for 24 hours.

3. Telemetry Payload Data Dictionary

All fields conform to the following schema. The backend parser natively supports both camelCase and snake_case keys.

Field Key	Type	Required	Units / Range	Description & Implementation Notes
`stationId`	String	YES	e.g. "ST-01"	Unique hardware node ID matching station label / barcode.
`eventType`	String	YES	"15min_triggered_event" | "daily_heartbeat"	Determines whether packet is an active detection or sleep keepalive.
`batteryVoltage`	Float	YES	3.30 to 4.25 (V)	Measured ADC battery voltage under moderate load.
`batteryLevel`	Integer	YES	0 to 100 (%)	Battery charge percentage estimated via lookup table / fuel gauge.
`soilMoisture`	Float	YES	0.0 to 100.0 (%)	Volumetric water content from capacitive moisture probe.
`acousticScore`	Integer	OPTIONAL	0 to 100 (or null)	Bioacoustic energy amplitude score. NULL in heartbeat mode.
`temperature`	Float	YES	°Celsius (e.g. 28.5)	Microclimate ambient temperature measured near base of tree trunk.
`humidity`	Float	YES	0.0 to 100.0 (%)	Relative air humidity measured by digital sensor (SHT31 / AHT20).
`isMoistureTriggered`	Boolean	YES	true | false	Flag indicating whether soil moisture exceeded activation threshold.
`mfccFeatures`	Array	OPTIONAL	13 Floats (or null)	13-band Mel-Frequency Cepstral Coefficients. NULL in heartbeat.
`rssi`	Integer	YES	-115 to -40 (dBm)	Cellular link signal strength reported by modem AT+CSQ.

4. Concrete JSON Payload Examples

4.1 Example: Active 15-Minute Triggered Packet (Acoustic Detection Active)

```
{
  "stationId": "ST-01",
  "eventType": "15min_triggered_event",
  "batteryVoltage": 3.92,
  "batteryLevel": 88,
  "soilMoisture": 72.5,
  "acousticScore": 84,
  "temperature": 29.1,
  "humidity": 83.4,
  "rssi": -76,
  "snr": 7.8,
  "isMoistureTriggered": true,
  "mfccFeatures": [
    -10.2, 14.5, 8.1, 6.3, 4.2, 3.1, 2.5, 1.9, 1.2, 0.8, 0.6, 0.4, 0.2
  ]
}
```

4.2 Example: 24-Hour Dormant Heartbeat Keepalive (Soil Dry, Sensor Gated Off)

```
{
  "stationId": "ST-01",
  "eventType": "daily_heartbeat",
  "batteryVoltage": 4.10,
  "batteryLevel": 97,
  "soilMoisture": 42.0,
  "acousticScore": null,
  "temperature": 27.8,
  "humidity": 65.2,
  "rssi": -80,
  "snr": 8.5,
  "isMoistureTriggered": false,
  "mfccFeatures": null
}
```

5. Server Response & Downlink Configuration Synchronization

Upon successful validation and database insertion, the server returns an HTTP 200 OK containing an acknowledgment and a nested 'config' object. The ESP32 firmware SHOULD inspect this object to dynamically adjust its local thresholds.

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "status": "SUCCESS",
  "message": "Telemetry packet accepted and processed",
  "eventId": "evt-1793173512813-686",
  "stationId": "ST-01",
  "serverTime": "2026-10-28 15:45:12",
  "config": {
    "moistureThreshold": 65.0,
    "samplingFrequencyMinutes": 15,
    "heartbeatIntervalHours": 24
  }
}
```

> **[FIRMWARE ADVANTAGE: PIGGYBACK DOWNLINK SYNC]** By parsing the 'config' block from the HTTP response, field nodes achieve dynamic OTA parameter synchronization (e.g., agronomy team alters threshold from 65% to 70% in the web UI) without establishing a power-hungry persistent MQTT downlink channel or waiting for SMS commands.

6. Firmware Implementation & Cellular Modem AT Sequence

For teams using raw AT commands via hardware UART on modems such as SIMCom SIM7600/A7670 or Quectel EC200U, the standard HTTP POST execution sequence is provided below:

```
// 1. Check Network Registration
AT+CGREG?                  --> Expect: +CGREG: 0,1 or 0,5 (Registered)
AT+CSQ                     --> Expect: +CSQ: <rssi>,<ber> (e.g. 24,99)

// 2. Set APN (Carrier Specific)
AT+CGDCONT=1,"IP","<YOUR_CARRIER_APN>"

// 3. Initialize HTTP Session
AT+HTTPINIT
AT+HTTPPARA="CID",1
AT+HTTPPARA="URL","http://<SERVER_IP>:8080/api/v1/telemetry/uplink"
AT+HTTPPARA="CONTENT","application/json"
AT+HTTPPARA="USERDATA","X-Node-API-Key: termiguard-node-secret-2026"

// 4. Send JSON Payload (Length in Bytes, Timeout in ms)
AT+HTTPDATA=<PAYLOAD_LENGTH>,10000
--> Wait for "DOWNLOAD" prompt, then write JSON payload bytes
--> Expect: "OK"

// 5. Trigger POST Action
AT+HTTPACTION=1            --> Expect: +HTTPACTION: 1,200,<RESPONSE_LENGTH>

// 6. Read Server Response & Config Sync
AT+HTTPREAD                --> Output response JSON
AT+HTTPTERM                --> Close HTTP session

// 7. Power down 4G Modem & Sleep ESP32
AT+CPOWD=1                 --> Graceful modem shutdown
esp_deep_sleep_start();    --> ESP32 enters deep sleep
```

[CRITICAL POWER PROTECTION: RETRY POLICY] If the cellular modem fails to register or the HTTP POST times out (e.g. tower congestion): (1) Do NOT retry continuously in a tight loop as this will rapidly deplete the LiSOCl2 battery; (2) Wait 60 seconds and attempt a single retry (maximum 3 attempts); (3) If connection still fails, save packet to SPIFFS flash memory and enter deep sleep until next cycle.